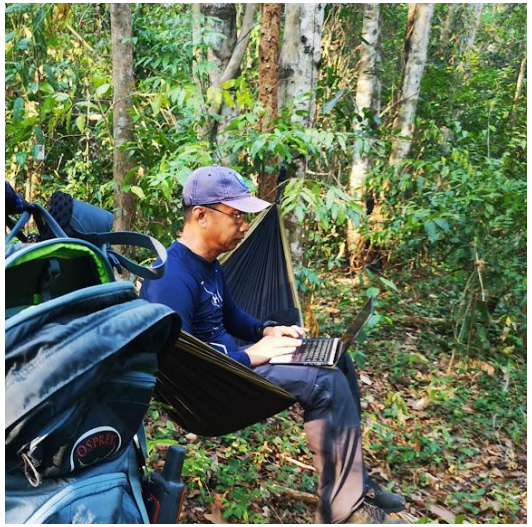
# ITCS214 : DATA STRUCTURES & ALGORITHM ANALYSIS



*Week 1 · Java foundations for building reliab*



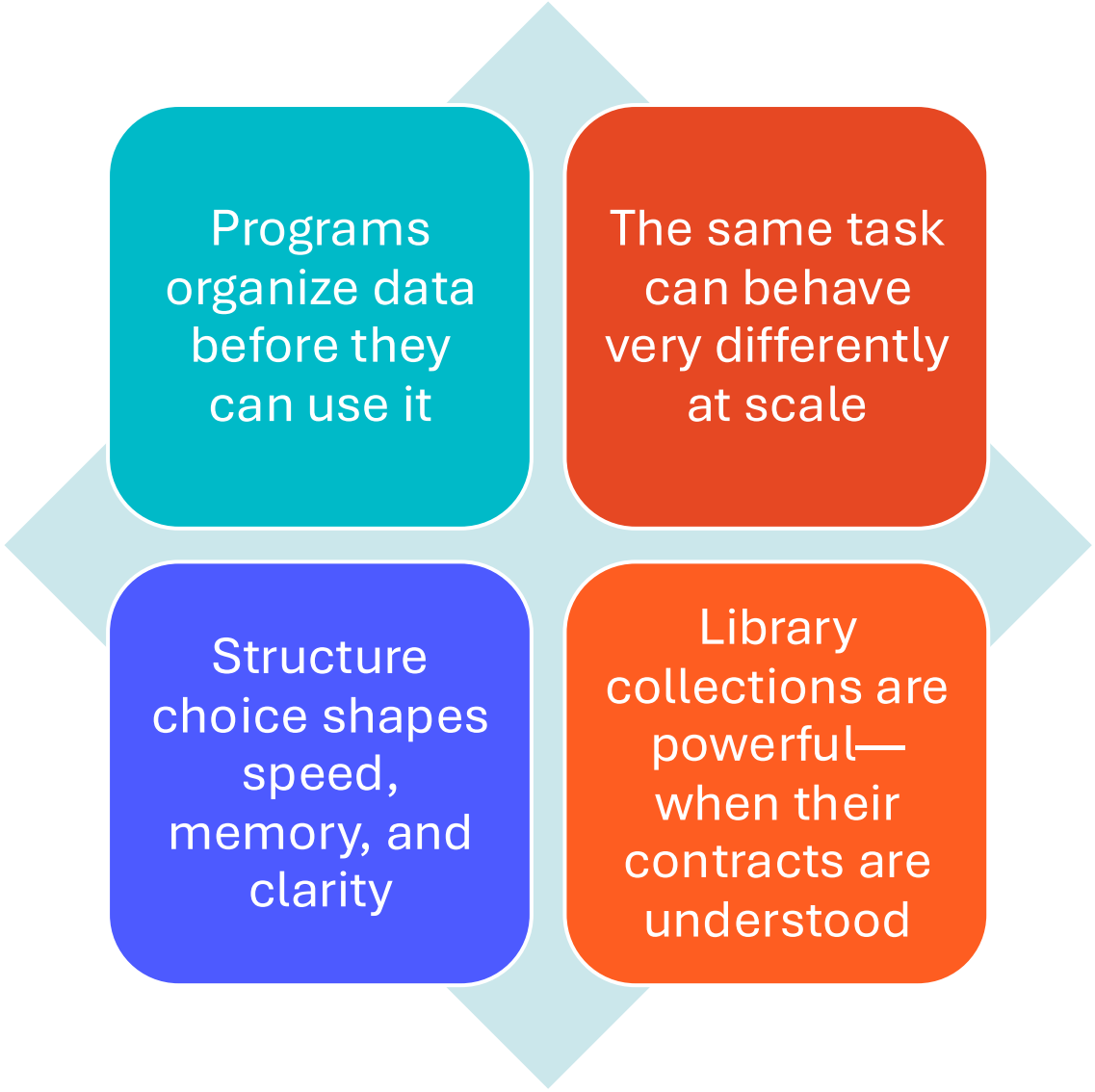
*Understand the contract. Protect the state. Test the behavior.*

Asst. Prof. Dr. Ananta Srisuphab
ananta.sri@mahidol.ac.th
2026/1

Mahidol University
Faculty of Information and Communication Technology

# Why this course matters?

## Correct code is only the starting point



# By the end of today...

**Explain**

- Data structure, algorithm, and ADT
- Class, object, reference, and invariant
- Identity, equality, and ordering

**Build & verify**

- A generic interface and implementation
- Boundary-focused JUnit 5 tests
- A disciplined AI-code review

# How we will work in this course

**:Reasoning is visible—not hidden inside code**

- Trace algorithms by hand
- Implement simplified structures in Java
- Test normal, boundary, and failure cases
- Explain design and complexity choices
- Use AI assistance transparently and verify its output

# What do you still remember?

*DIAGNOSTIC*

# Data and algorithms play different roles

## Data structure

- Organizes values
- Supports a defined operation set
- Carries representation choices

## Algorithm

- A finite sequence of steps
- Transforms inputs into outputs
- Consumes time and space

# A design decision flows from the problem




Printing jobs → first received, first processed → Queue → circular array or linked nodes

# An ADT specifies what; a class decides how

**Abstract behavior**

- Available operations
- Rules and outcomes
- Errors and edge cases

**Concrete implementation**

- Fields and memory layout
- Algorithms inside methods
- Performance trade-offs

# No single structure wins every task

**Choose from the operations that dominate**

- Student lookup by ID
- Alphabetical listing
- Registration-order processing
- Frequent insertion or removal
- Memory, ordering, duplicate, and persistence constraints

# One ADT can have several implementations

| ADT | Possible representations |
| --- | --- |
| • List | • Dynamic array / linked list |
| • Stack | • Array / linked nodes |
| • Queue | • Circular array / linked nodes |
| • Set | • Hash table / balanced tree |
| • Priority queue | • Heap / sorted list |

# A class defines; an object exists

## Class

- A type definition
- Declares fields and methods
- Describes valid construction

## Object

- A runtime instance
- Has current state
- Reached through a reference

# References can alias the same object

```java
class Student {

    String id;
    String name;

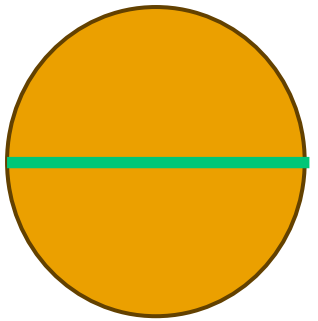
    Student(String id, String name) {
        this.id = id;
        this.name = name;
    }

    void setName(String name) {
        this.name = name;
    }
}
```

```java
public class StudentTester {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        Student first = new Student("001", "Mali");
        Student second = first;

        second.setName("Suda");

        System.out.println(first.name);  // Suda
        System.out.println(second.name); // Suda
    }
}
```



**PREDICT**

What does first.getName() return?

# Encapsulation protects invariants

**Private representation**

- Clients cannot mutate fields directly
- Methods become controlled entry points
- Internal representation can change

**Invariant**

- A property true in every valid state
- Established by constructors
- Preserved by every public method

# Constructors establish a valid starting state

```java
public class Counter {
    int value;
    public Counter(int initialValue) {
    if (initialValue < 0) {
        throw new IllegalArgumentException(
            "initialValue must be non-negative");
    }
    this.value = initialValue;
    }
}
```

**CONTRACT**

Precondition: initialValue $\geq 0$
Postcondition: value == initialValue

**COMMON MISTAKE**

Assign first, validate later.

# An interface is a behavioral contract

```java
public interface Stack<T> {
    void push(T item);
    T pop();
    T peek();
    boolean isEmpty();
}
```

```java
public class ArrayStack<T> implements Stack<T> {

    private T[] stack;
    private int top;
    private static final int DEFAULT_SIZE = 10;

    @SuppressWarnings("unchecked")
    public ArrayStack() {
        stack = (T[]) new Object[DEFAULT_SIZE];
        top = -1;
    }

    @Override
    public void push(T item) {
        if (top == stack.length - 1) {
            System.out.println("Stack Overflow");
```

```java
public class StackTester {

    Run | Debug | Run main | Debug main
    public static void main(String[] args) {

        Stack<Integer> stack = new ArrayStack<>();

        System.out.println("Is Empty? " + stack.isEmpty());

        stack.push(10);
```

**POLYMORPHISM**

Stack<String> s = new ArrayStack<>();

Later, the implementation can change while client code keeps the same contract.

# Object-based containers defer mistakes to runtime

## Without generics

- Values stored as Object
- Clients cast results
- Wrong casts fail at runtime

## With generics

- Element type is explicit
- Compiler rejects mixed types
- Fewer casts; clearer intent

# A type parameter becomes part of the contract

Box.java > ...

```java
public class Box<T> {

    private T value;

    public Box(T value) {
        this.value = value;
    }

    public T getValue() {
        return value;
    }

    public void setValue(T value) {
        this.value = value;
    }

    @Override
    public String toString() {
        return "Box[" + value + "]";
    }
}
```

```java
public static void main(String[] args) {

    // Box of String
    Box<String> message = new Box<>("Hello");

    System.out.println(message);
    System.out.println("Message = " + message.getValue());

    message.setValue("Welcome to Java");
    System.out.println("After update = " + message.getValue());

    System.out.println();
```

**COMPILER GUARD**

score.setValue("ninety-five"); // rejected

```java
    Box<Double> temperature = new Box<>(36.5);

    System.out.println(temperature);

    temperature.setValue(37.2);

    System

    System

    // Com
    // Unc

    score.setValue("Ninety Five");
}
```

```
The method setValue(Integer) in the type Box<Integer> is not applicable for the arguments
(String) Java(67108979)

incompatible types: String cannot be converted to Integer (errors(1): 41:24-41:37)

void Box.setValue(Integer value)

Source: ITCS214_b04bdf04

View Problem (⌥F8)   Quick Fix... (⌘.)
```

# Raw types throw away useful guarantees

**Parameterize every generic use**

- Avoid: Container values
- Prefer: Container<Student> values
- The diamond <> lets Java infer constructor arguments
- Bounded parameters are possible: <T extends Comparable<T>>

# == asks identity; equals() asks logical equality

```
Student a = new Student("001", "Mali");
Student b = new Student("001", "Mali");

System.out.println(a == b);      // false
System.out.println(a.equals(b)); // true
```

**DESIGN CHOICE**

Here, student ID defines logical equality.

**PREDICT**

What if b = a?

# Equal objects must share a hash code

```
@Override
public boolean equals(Object other) {
    if (this == other) return true;
    if (!(other instanceof Student s)) return
false;
    return id.equals(s.id);
}

@Override
public int hashCode() { return id.hashCode(); }
```

- Override both methods together
- Equal ⇒ same hash code
- Unequal objects may still collide
- Hash tables rely on this contract

# Ordering can be natural or contextual

## Comparable<Student>

- Implemented by the class
- One natural ordering
- compareTo(other)

## Comparator<Student>

- Separate strategy
- Multiple alternative orderings
- by name, score, or other key

# Student: equality by ID, ordering by context

```
@Override
public int compareTo(Student other) {
    return id.compareTo(other.id);
}

Comparator<Student> byScore =
    Comparator.comparingDouble(Student::getScore)
            .thenComparing(Student::getName);
```

**KEY DISTINCTION**

Equality answers “same logical entity?”
Ordering answers “which comes first?”

# Fail fast at the boundary

Choose exceptions that describe the violated contract

- IllegalArgumentException · bad constructor/method argument
- IndexOutOfBoundsException · invalid logical index
- IllegalStateException · operation impossible in current state
- NullPointerException · required reference is null

# Java offers interfaces and multiple implementations

| | |
|---|---|
| **List** | ArrayList · LinkedList |
| **Set** | HashSet · TreeSet |
| **Queue / Deque** | ArrayDeque · PriorityQueue |
| **Map** | HashMap · TreeMap |

PRACTICE

Use library collections—and implement simplified versions to understand their behavior.

# Our central example: a fixed-capacity generic container.

# The interface states the observable behavior

```
public interface Container<T> {
    void add(T item);
    T get(int index);
    boolean remove(T item);
    boolean contains(T item);
    int size();
    boolean isEmpty();
    void clear();
}
```

# Two fields encode the representation

```
public class FixedArrayContainer<T>
        implements Container<T> {
    private final T[] elements;
    private int size;
}
```

**REPRESENTATION INVARIANT**

elements != null
0 ≤ size ≤ elements.length
active items occupy [0, size)

**FINAL MEANS**

The array reference cannot be reassigned; its elements can still change.

# Construction validates capacity and creates storage

```
@SuppressWarnings("unchecked")
public FixedArrayContainer(int capacity) {
    if (capacity <= 0) {
        throw new IllegalArgumentException(
            "capacity must be greater than zero");
    }
    elements = (T[]) new Object[capacity];
}
```

**WHY THE CAST?**

Java cannot directly create new T[capacity] because generic type information is erased.

# Constant-work methods expose safe observations

```
@Override
public int size() {
   return size;
}


@Override
public boolean isEmpty() {
   return size == 0;
}
```

**QUESTION**

Why return size instead of counting non-null slots?

**INVARIANT AT WORK**

The field is trustworthy only because every mutation preserves it.

# add() protects both the policy and the capacity

```
@Override
public void add(T item) {
    Objects.requireNonNull(item, "item");
    if (size == elements.length) {
        throw new IllegalStateException("container is full");
    }
    elements[size++] = item;
}
```

- Reject null
- Fail before writing when full
- Write at the first inactive slot
- Advance logical size

# get() validates against logical size—not capacity

```
@Override
public T get(int index) {
    checkIndex(index);
    return elements[index];
}

private void checkIndex(int index) {
    if (index < 0 || index >= size) {
        throw new IndexOutOfBoundsException(
            "index: " + index + ", size: " + size);
    }
}
```

**BOUNDARY**

When size = 2, index 2 is invalid even if capacity = 10.

# contains() searches only active elements

```
@Override
public boolean contains(T item) {
    for (int i = 0; i < size; i++) {
        if (Objects.equals(elements[i], item)) {
            return true;
        }
    }
    return false;
}
```

EQUALITY

Objects.equals() applies logical equality and safely handles a null search key.

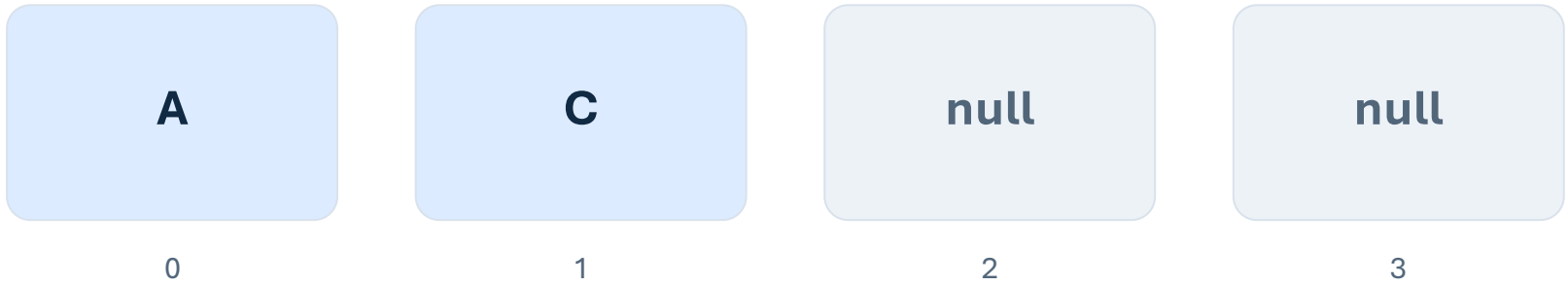
# remove() repairs the contiguous active region

```
@Override
public boolean remove(T item) {
    for (int i = 0; i < size; i++) {
        if (Objects.equals(elements[i], item)) {
            for (int j = i; j < size - 1; j++)
                elements[j] = elements[j + 1];
            elements[--size] = null;
            return true;
        }
    }
    return false;
}
```

THREE JOBS

1 · find
2 · shift left
3 · clear old last slot

# Logical contents occupy [0, size)




**size = 2**

**capacity = 4**

PHYSICAL VERSUS LOGICAL

The array has four slots; clients can access only the first two.

# clear() releases every active reference

```
@Override
public void clear() {
    Arrays.fill(elements, 0, size, null);
    size = 0;
}
```

**POSTCONDITION**

size == 0
isEmpty() == true
capacity is unchanged

# Arrange → Act → Assert

## Test structure

- Arrange a known state
- Act once when possible
- Assert observable behavior

## Test categories

- Normal case
- Boundary case
- Error case
- Repeated operations

# JUnit expresses behavior in small, focused tests

```
@Test
void addedItemCanBeRetrieved() {
   Container<String> c =
      new FixedArrayContainer<>(3); // arrange

   c.add("A");                   // act

   assertEquals("A", c.get(0));     // assert
   assertEquals(1, c.size());
}
```

- Name the behavior
- Use the public contract
- Keep failure messages local
- Prefer independent tests

# Exceptions are part of the contract

```
@Test
void addingToFullContainerFails() {
   Container<String> c =
      new FixedArrayContainer<>(1);
   c.add("A");

   assertThrows(IllegalStateException.class,
      () -> c.add("B"));
}
```

**ALSO TEST**

Invalid constructor capacity
Invalid get index
Null insertion

# Design tests around transitions

The most revealing cases change the structure’s shape

- Empty → one item
- One free slot → full
- Remove first, middle, and last
- Remove an absent item
- Clear empty and non-empty containers
- Equal-but-distinct objects

AI VERIFICATION

Compile, inspect, test, analyze, revise, and document.

# Verification is a repeatable workflow




**Document assistance and explain every submitted line.**

# Find the bugs before the tests do

```
public boolean remove(T item) {
    for (int index = 0; index <= size; index++) {
        if (elements[index] == item) {
            elements[index] = null;
            size--;
            return true;
        }
    }
    return false;
}
```

- Loop boundary?
- Equality rule?
- Shift required?
- Invariant preserved?
- Which tests expose it?

# Use evidence to audit AI output

## Inspect

- Matches specification
- Preserves invariants
- Uses suitable exceptions
- Has coherent equality semantics

## Test & explain

- Boundary cases pass
- Complexity claim matches loops
- Every line is explainable
- Assistance is disclosed

# Some operations stay flat; others scan or shift

| | | |
|---|---|---|
| **size()** | return one field | **does not grow** |
| **get(index)** | one checked access | **does not grow** |
| **add(item)** | one write when space exists | **does not grow** |
| **contains(item)** | may inspect every active item | **grows with size** |
| **remove(item)** | search + possible shifts | **grows with size** |
| **clear()** | clears every active reference | **grows with size** |